

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 4 — Un agente nel repository

La rete di sicurezza

Auto mode: chi decide

Un classificatore guarda l’intento, non se il codice funziona

- Nel Modulo 4, dopo i permessi: che cosa vi protegge quando Claude lavora **con meno domande**. Quattro reti: auto mode, il sandbox, i checkpoint, gli hook
- In **auto mode** un **classificatore**, un modello separato, esamina le azioni prima che partano, e blocca ciò che va **oltre la vostra richiesta**. C’è su tutti i piani, e dal 14 agosto 2026 le nuove sessioni sul Pro partono così
- Di serie blocca, per esempio, codice scaricato ed eseguito (`curl | bash`), deploy e migrazioni in produzione, force push. Dalle sue richieste si tolgono i **risultati degli strumenti**: un contenuto ostile in un file o in una pagina web non lo manipola direttamente
- Riduce le richieste di permesso, ma **non garantisce la sicurezza**, e non sostituisce un’attenta revisione umana sull’infrastruttura ad alto rischio
- Controlla l’**intento**, non la correttezza: non si accorge se il codice funziona davvero. Per questo, nel corso «Claude Code in action» dell’Academy, lo si abbina a uno **Stop hook** che esegue i test
- I numeri di Anthropic: gli utenti approvano il **97 %** delle richieste di permesso, e con il tempo si smette di prestare molta attenzione a ciò che si approva. In un esperimento con 1.053 tester, la revisione umana ha fermato il 13,6 % dei comandi pericolosi, auto mode l’**89 %**; su azioni reali troppo zelanti, il classificatore ha il 17 % di **falsi negativi** (marzo e agosto 2026)

Il sandbox

Un confine del sistema operativo, per i soli comandi di shell

- Un confine che il **sistema operativo** impone attorno ai comandi di shell di Claude, **solo a quelli**: gli strumenti per i file, i server MCP e gli hook girano fuori
- Su macOS, Linux e WSL 2; **spento** di serie, si accende con `/sandbox`. Su Linux e WSL 2 vuole due pacchetti di sistema, `bubblewrap` e `socat`, da installare con `sudo apt-get` su Ubuntu e Debian: qui lo spieghiamo, **non lo installiamo**
- Dentro, i comandi **scrivono** solo nella cartella di lavoro, in una temporanea e in quelle aggiunte, ma **leggono** quasi tutta la macchina, credenziali comprese; la rete passa da un proxy, con una lista di domini consentiti che parte vuota
- Se manca una dipendenza, di serie i comandi girano **senza sandbox**. Riduce il rischio, ma non isola del tutto; nell'uso interno di Anthropic ha tolto l'**84 %** delle richieste di permesso

Checkpoint e rewind

Tornare indietro: che cosa torna, e che cosa no

- Prima di modificare un file, Claude ne salva il contenuto: un **checkpoint**. Ogni vostro prompt ne crea uno a cui tornare
- Il menu del rewind: **doppio Esc** sul prompt vuoto. Tre ripristini: codice e conversazione insieme; la sola conversazione, tenendo il codice com'è; il solo codice, tenendo la conversazione
- Un file cambiato **solo da comandi Bash** non torna: per quello, **git**. E dalla 2.1.309 `sed -i` è un comando Bash come gli altri: come loro, `/rewind` non lo ripristina
- Torna invece ciò che un comando Bash cambia **dopo**, in un file che Claude aveva già modificato con i suoi strumenti
- Le modifiche di un **subagent**, di solito, nei checkpoint non entrano
- Le azioni su **sistemi remoti** — database, API, deploy — non si annullano con i checkpoint
- E **git**: i team di Anthropic partono da uno stato pulito, e fanno spesso commit di checkpoint, per annullare le modifiche sbagliate

Gli hook

Comandi che Claude Code esegue in punti precisi del suo ciclo

- Comandi di shell, scritti da voi, che **Claude Code** esegue in punti precisi del suo ciclo: un controllo **deterministico**, che vale qualunque cosa decida il modello
- Tre cadenze: per **sessione** (SessionStart, SessionEnd), per **turno** (UserPromptSubmit, **Stop**), per **chiamata** a uno strumento (PreToolUse, PostToolUse)
- Sugli eventi che possono bloccare, un hook che esce con **2** blocca l'azione: un PreToolUse blocca la chiamata allo strumento. Secondo l'evento, ciò che scrive su stderr torna a Claude. Un PreToolUse che esce con 0 non approva nulla: la chiamata passa comunque dai permessi. E il `deny` di un hook vale anche in `bypassPermissions`
- `/hooks` li elenca, per evento. Girano con **tutti i vostri permessi**
- Nel comando, `$CLAUDE_PROJECT_DIR` è la radice del progetto in cui la sessione è cominciata: la usa la demo che segue

M4-D05 Il controllo che non si salta

Demo del docente in Claude Code: lo stesso prompt, con e senza Stop hook

- Due bracci che differiscono in **una cosa sola**: lo **Stop hook**, nelle impostazioni della copia del progetto. Lo stesso prompt chiede **soltanto** una docstring più chiara; ma nel progetto delle urgenze il codice dice `> 7`, la regola «almeno 7 giorni», e un test è rosso

`prompt.txt`, uguale nei due bracci

```
Riscrivi la docstring di urgenza.py in modo più chiaro, con un
esempio d'uso.
```

- Nel braccio con l'hook, in `.claude/settings.json` della copia, all'evento `Stop`, un hook di tipo `command`:

```
cd "$CLAUDE_PROJECT_DIR" && python3 -m unittest -q 1>&2 || exit 2
```

- Esegue i test, manda la loro uscita su stderr e, se falliscono, esce con **2**. `claude -p` parte dalla copia: Sonnet, effort low, acceptEdits, gli strumenti Read, Edit e `Bash(python3 -m unittest*)`
- Guardate quante volte l'hook **blocca** la fine del turno, che cosa cambia nella copia, e i **test** alla fine

M4-D05 Il controllo che non si salta

Demo del docente in Claude Code: lo stesso prompt, con e senza Stop hook

- Dalla pagina delle best practice di Claude Code, sullo Stop hook: «Exit code 2 is what blocks the stop. Whatever the command prints to stderr goes back to Claude as the reason it can't finish yet». Una **garanzia** del prodotto: deve valere a ogni esecuzione
- Nella stessa pagina, con il compilatore di TypeScript come controllo: «ask Claude for a change that breaks a type: instead of finishing, it gets the compiler errors back and keeps going until tsc passes». Un **comportamento** descritto: si conta su quante esecuzioni. Qui il controllo sono i test, e il prompt non li nomina
- E il limite: «If the hook blocks eight stops in a row with no tool call between them, Claude Code overrides the next block and ends the turn.»
- Il controllo dell'elemento legge lo stream di ogni esecuzione: conta i **blocchi** dell'hook, dice quali file della copia sono **cambiati**, e **riesegue i test** nella copia

M4-D05 Il controllo che non si salta

Demo del docente in Claude Code: lo stesso prompt, con e senza Stop hook

La registrazione del corso: 3 esecuzioni per braccio, ognuna in una copia nuova del progetto; poi il controllo dell'elemento, sullo stream e sulla copia di ognuna

esecuzione	blocchi dello Stop hook	i test, rieseguiti ora nella copia
con-hook-1	1 blocco	verdi (3 test, OK)
con-hook-2	1 blocco	verdi (3 test, OK)
con-hook-3	1 blocco	verdi (3 test, OK)
senza-hook-1	0 blocchi	rossi (3 test, FAILED (failures=1))
senza-hook-2	0 blocchi	rossi (3 test, FAILED (failures=1))
senza-hook-3	0 blocchi	rossi (3 test, FAILED (failures=1))

Le righe del controllo, in tabella
Registrazione del corso, 12 novembre 2026, Claude Code 2.1.322, Sonnet

- Con l'hook, «L'uscita 2 dello Stop hook blocca la fine del turno»: confermata in tutte e tre, «1 blocco, e il turno si è chiuso con i test verdi». «Claude continua finché il controllo passa»: confermata 3 su 3
- In tutte e tre, «cambiati: urgenza.py; invariati: test_urgenza.py»: il confine è stato corretto nel codice, anche se il prompt chiedeva soltanto la docstring
- Senza l'hook: «Senza lo Stop hook, i test verdi alla fine: 0 su 3». Senza un controllo, dice Anthropic, Claude si ferma quando il lavoro sembra finito
- Un'istruzione nel prompt, Claude può anche non seguirla; un hook, Claude Code deve eseguirlo

Prima di lasciarlo andare da solo

Cinque domande di Anthropic, e una per voi

Prima di un'esecuzione non sorvegliata, la pagina delle best practice di Claude Code fa cinque domande:

1. Sa com'è il lavoro **finito**?
2. Può **controllare** il proprio lavoro?
3. Che cosa può **toccare**?
4. Che cosa lo **ferma**? Per esempio `--max-turns`: in `claude -p` limita i turni; di serie, un limite non c'è
5. Dove va il **risultato**, e chi lo **approva**? Una PR, un commento o un riassunto; poi una persona che approva il merge, il deploy o ciò che non si può annullare

Un'esecuzione headless, con `claude -p`, si verifica dal suo **JSON** e dal **codice d'uscita**

Lo Stop hook ha un limite: se blocca la fine del turno **otto volte di fila**, senza chiamate a strumenti in mezzo, Claude Code scavalca il blocco successivo e chiude il turno

A voce: che cosa vi farebbe lasciare lavorare Claude da solo per un'ora?

Un controllo imposto dall'harness non si salta: Claude finisce quando i test passano, non quando il lavoro gli sembra finito.